Butterfold

Folding all the way

Track: D (AI / LLM Assisted Circuits)

Team: D3 - ButterFold

Process: GF180MCU

Contents

1. Introduction

This document describes the architecture of the ButterFold mixed-radix OFDM transform engine, designed for the GF180MCU process as part of the Chipathon 2025 Track D. The system implements a folded FFT architecture that reuses a single butterfly unit to perform 12-pt and 128-pt DFTs, targeting 5G NR PHY resource element mapping.

2. Motivation

5G NR OFDM requires mixed-radix transforms (12-pt and 128-pt) that a conventional FFT accelerator can't handle efficiently. ButterFold's folded architecture replaces 2x the area with a single butterfly unit and an SRAM buffer.

3. Theory

3.1. Number Representation

The numbers in the system are represented in ARM's variant of Q1.7 fixed-point format. All arithmetic operates in two's complement; the Q-format only determines the position of the binary point.

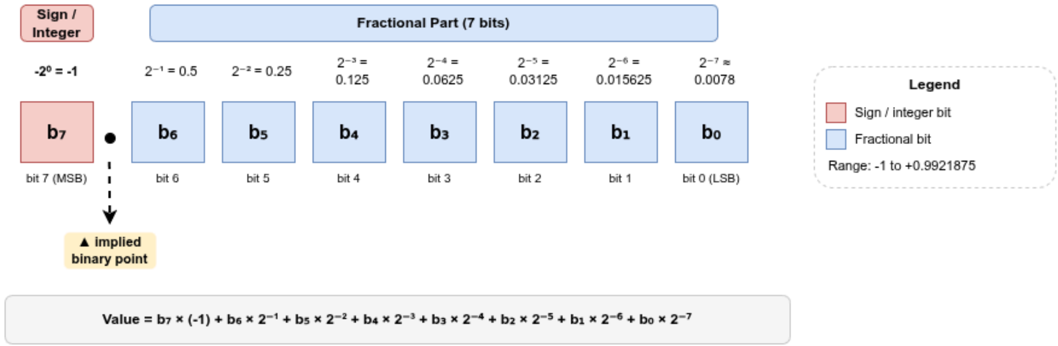

Figure 1: Q1.7 Fixed-Point Format - Bit Breakdown

3.1.1. Value Range

The largest positive number representable with 8-bits is $2^{8-1} - 1 = 127_{10}$ 2s complement equivalent to 01111111_2. In Q1.7 notation it would be $0 + 2^{-1} + \ldots + 2^{-7} = 0.9921875$.

Similarly, the smallest negative number representable is

$$-2^{8-1} = -128_{10}$$

2s complement equivalent to 10000000_2. In Q1.7 notation it would be $2^0 = -1$.

3.2. Operating on Q-formats

Adding or subtracting two Q1.7 numbers is the same as operating on two 8b numbers; the result is 9b and is represented in Q2.7 notation. The result is usually stored as Q1.8 to give additional fractional precision. Q(m,n)

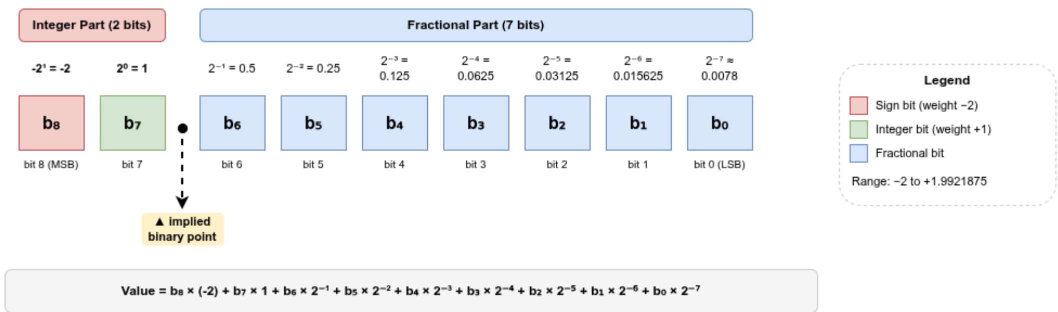

Figure 2: Q2.7 Fixed-Point Format - Bit Breakdown

It has a value range of $[-2.0, +1.96875]$.

Multiplying two Q1.7 numbers results in a 16b number in Q2.14. In many cases the MSB is redundant, so it is represented as Q1.15 for additional fractional precision.

Dividing two Q-format numbers is more involved. Dividing two 8b integers generally produces a result <= 8b, but in Q-format the result retains the same fractional bits while gaining additional integer bits to account for the range.

3.3. Converting back to Q1.7 (Quantisation)

The steps are:

- `shift = <source fractional bits> - <target fractional bits>`
- Add bias of $2^{\text{shift}-1}$
  - this is optional; omitting it creates a bias toward zero
  - bias reduces the error and brings the result closer to the true value
  - only add bias when shift > 0
- If shift >= 0, shift the source number right (>>) by `shift`;
  otherwise, shift the source number left (<<) by `shift`
- Saturate the value to fit in the target representation's range
  - hardware optimisation

MSB	MSB-1	Description
0	0	no overflow
0	1	clamp to MAX
1	0	clamp to MIN
1	1	no overflow

With Q1.7/Q1.8 arithmetic defined, we can now look at the system that uses it.

4. Proposed Architecture

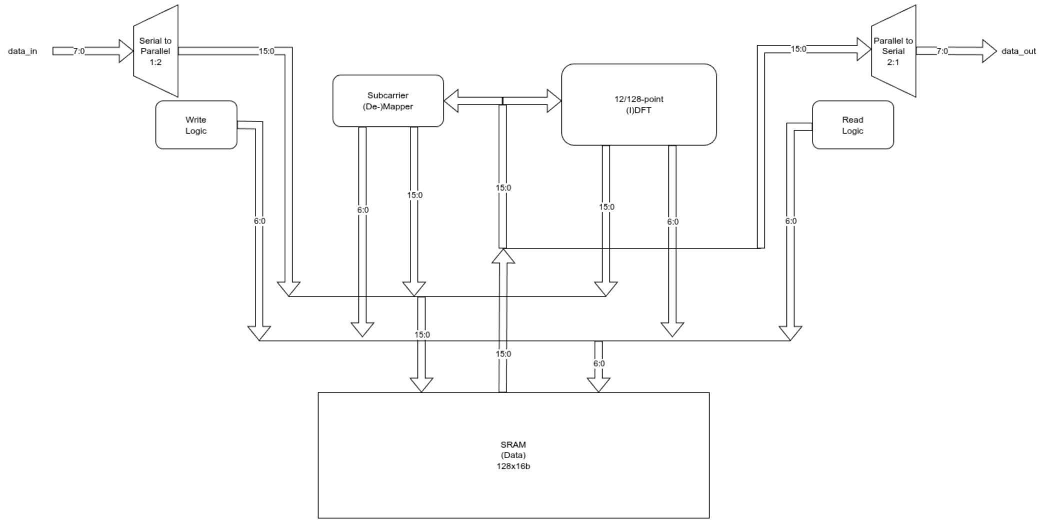

Figure 3: Folded PHY Top Level Architecture

4.1. Data Flow

4.1.1. Tx

1. The Write Logic stores the input data in SRAM.
2. The 12pt FFT operates on the first 12 elements in SRAM and stores the result back.
3. The Subcarrier Mapper offsets the data in SRAM and assigns zero to all other elements for the 128pt FFT.
4. A 128pt FFT is performed on the first 128 elements of the SRAM.
5. The Read Logic reads out the data at a pace the parallel to serial block can receive, reading the last N_CP elements first and then the first 128 elements.

4.1.2. Rx

1. The Write Logic rejects the first N_CP valid input samples and stores the incoming 128-element stream in SRAM.
2. The 128pt IFFT operates on the first 128 elements in SRAM and stores the output back in order.
3. The Subcarrier De-mapper extracts 12 elements from SRAM at a given offset.
4. The Read Logic reads out the data at a pace the parallel to serial block can receive, in the same order as stored.

4.2. FFT

The FFT block performs 12-pt FFT, 128-pt FFT, and 128-pt IFFT by serialising the operation through a single butterfly structure. The same structure may be reused for both Radix-2 and Radix-3 computations. It reads data from memory, computes the transform, and stores the result back in the same memory. The block operates autonomously from start to end using an internal state machine.

Signal Name	I/O	Description
clk	Input	
rst	Input	
mode	Input	to switch between 12pt and 128pt FFT mode
inverse	Input	to produce an IFFT output
start	Input	Assert for 1 cycle to begin FFT

Signal Name	I/O	Description
data_addr[6:0]	Output	
data_in[16:0]	Input	
data_in_valid	Input	
twiddle_addr[6:0]	Output	
twiddle_in[16:0]	Input	
twiddle_in_valid	Input	
data_out[16:0]	Output	
data_out_valid	Output	
busy	Output	High while FFT is in progress
almost_done	Output	Final operation indicator
overflow	Output	Debug - Arithmetic overflow occurred
saturation_occurred	Output	Debug - One or more values were saturated

- Using prime factorisation to get the smallest radix core needed for computing a certain FFT.
- Using Q1.8 format for the stored data and logic to quantise the computed data before storing back to memory.

4.3. Serial to Parallel

Packs incoming frequency-domain IQ data from 8-bit time-multiplexed I and Q samples into 16-bit words (8b I, 8b Q).

Signal Name	I/O	Description
clk	Input	
rst	Input	
data_in[7:0]	Input	
data_in_valid	Input	data_in_valid of the core
data_out[15:0]	Output	
data_out_valid	Output	used as wr-enable & addr increment for Write Logic

4.4. Write Logic

Works with the Serial to Parallel block and sets the SRAM address for incoming data. In Tx mode, 12 samples are stored. In Rx mode, the block discards N_CP*2 samples (each number is made up of 1 complex & 1 real component) before storing the next 128 samples.

Parameter Name	Description
N_CP	Number of initial samples to ignore, in Rx mode

Signal Name	I/O	Description
clk	Input	
rst	Input	
mode	Input	
wr	Input	SRAM write enable
data_addr[6:0]	Output	

There is a concern about timing between the SerToPar block asserting valid and the address increment. Care must be taken with the SRAM write enable, since the line transitions every alternate cycle.

4.5. Parallel to Serial

Unpacks 16-bit SRAM data to an 8-bit Time-domain IQ data stream.

Signal Name	I/O	Description
clk	Input	
rst	Input	
data_in[7:0]	Input	
data_in_valid	Input	
data_out[15:0]	Output	
data_out_valid	Output	data_out_valid of the core

4.6. Read Logic

Works with the Parallel to Serial block and sets the SRAM read address sequence. In Tx mode, 128 samples are read out. In Rx mode, the block reads the last N_CP addresses first, then continues with the remaining data to produce 12 samples.

Parameter Name	Description
N_CP	Number of initial samples to read, in Rx mode

Signal Name	I/O	Description
clk	Input	
rst	Input	
mode	Input	
data_out_valid	Input	Used to know when to activate rd enable of the SRAM
data_addr[6:0]	Output	
rd	Output	

4.7. Subcarrier Mapper / De-mapper

In Tx, the block maps the 12pt DFT output into 128 frequency bins. In Rx, it de-maps 12 samples from 128 frequency bins.

Parameter Name	Description
OFFSET	amount to shift the data values by

Signal Name	I/O	Description
clk	Input	
rst	Input	
mode	Input	
start	Input	
busy	Output	
wr_rd_n	Output	When not reading must always be de-asserted

Signal Name	I/O	Description
data_in[15:0]	Input	
data_in_addr[6:0]	Output	
data_out_addr[6:0]	Output	
data_out[15:0]	Output	

In Tx, the block shifts the 12 samples in the SRAM by OFFSET and assigns zero to all other elements in the 128-sample array. In Rx, the block moves the 12 samples starting at the OFFSET address in the SRAM to address 0.

4.8. Global Controller

It is assumed all components inside the core operate at the same clock frequency. The global controller is responsible for issuing start pulses to each block in the sequence specified by the OFDM-s-DFT 5G standard for signal transformation. This is necessary because the SRAM is a shared resource; when one block accesses it, it is actively read from and written to, leaving no room for pipelining with other operations. The operation sequence is described in the Data Flow section.

4.9. Test Logic

Post-silicon validation is critical for confidence in the design under real-world conditions. Due to a shortage of pins in the padframe, instructing the core to enter a debug state through dedicated pins is not possible. Instead, pre-existing pins are multiplexed for this purpose. The `mode` and `busy` lines have been identified for this task.

During normal operation, `mode` selects Tx or Rx mode and `busy` goes high once all data for that mode has been loaded (12 samples for Tx, 128 for Rx). For debugging, `mode` acts as a MOSI line: a processor toggles it to transmit an 8-bit message, and an internal sequence detector decodes these toggles to enter debug mode. The `busy` line acts as MISO, outputting status messages as test cases are cycled through.

5. Chipathon Constraints

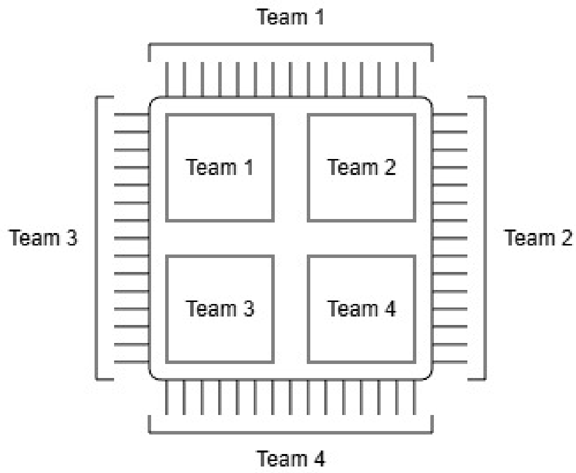

Figure 4: Team level allocation of the die

5.1. Budgeting

Field	Parameters $_{(max)}$	Description
Power	5V × 0.5A	Not confirmed
Area	3.05mm × 1.6mm	Not confirmed
Pins	22 + 1(VDD) + 1(GND)	Padframe A
F_{clk}	150MHz	

Table 1: Process constraints

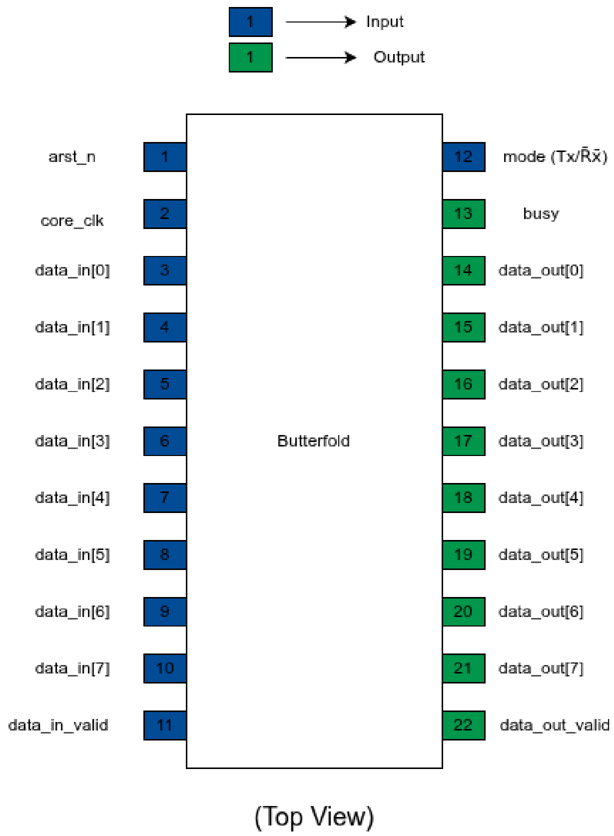

Figure 5: Expected pinout

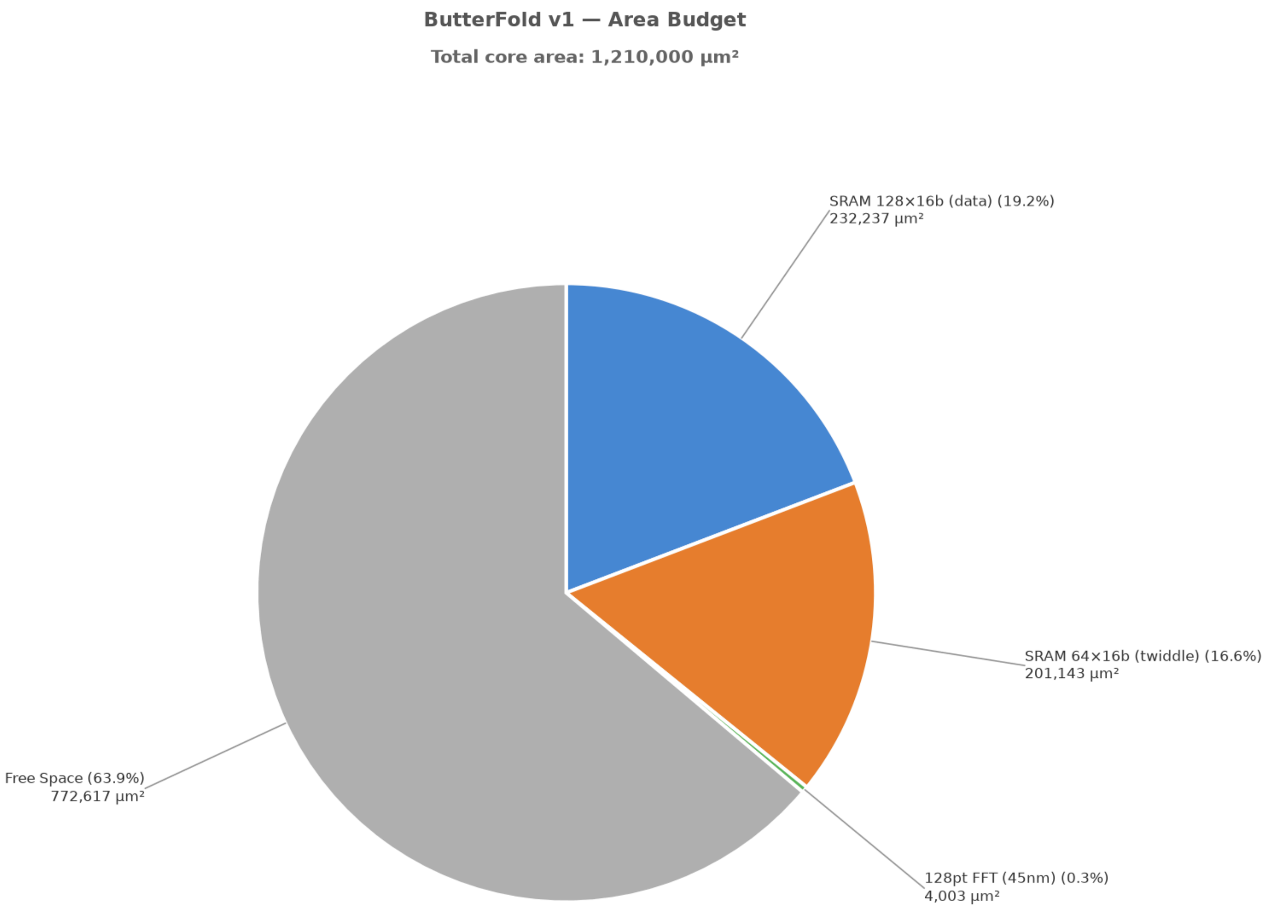

Figure 6: Area budget

LibreLane standard cells operate at 5.0V (confirmed by inspecting the macros in the docker container). SRAM can also be configured to use 5.0V.